

Report: Adversarial Random Forest and CTGAN Peptide Data Synthesis

Atomic Intelligence R&D team

1 Introduction

In this report, we outline our work, and thought process when generating synthetic data with Adversarial Random Forests (ARFs), Generative Adversarial networks, and Histogram based Imputation. We provide high-level overview on how each method works, and justifications for the methods used.

2 Adversarial Random Forest

Adversarial Random Forests are a method of generating synthetic data that uses Random Forest Classifiers to find regions of the feature space where the data are distributed independently. Let $\mathcal{D} = \{X_i | i = 1 \dots N\}$ be the real data set that we want to mimic. Each example X_i may be a combination of a numeric part $x^{(n)} \in \mathbb{R}^d$ and categorical $x^c \in \mathbb{Z}^C$, in practice we may want to represent X_i as a concatenated vector, e.g., $(x^{(n)}, x^c)$. The main take-away is that the examples might contain both numerical and categorical features, and the algorithm will work, regardless of the representation. For simplicity sake, we will carry out the following description assuming that our data contains only numerical features.

The model fitting proceeds as follows. First, we assume that the underlying joint distribution is already independent, i.e. each feature does not depend on other features. More formally

$$p(x) = \prod_{k=1}^d p(x^d)$$

Here x^d is the d -th component of the feature vector x . Next, we sample N synthetic examples from the 'independent' distribution. Obviously these data will fail to resemble the real data but now we train a Random Forest Classifier to discriminate between real and synthetic data.

Once a Random Forest Classifier has been properly trained, it is able to divide the feature space into regions where real data should be present and where synthetic data should be present, in fact these regions are called random tree leafs and geometrically they are hyperrectangles. In 2D these are simply rectangles.

Here it usually happens that the classifier achieves 90% accuracy or higher, however, now we make adjustments to our data generation step. First we randomly sample a leaf (or rectangle) from our classifier. The probability of choosing a certain leaf is proportional to the data that leaf covers. E.g. if we have one random tree with two leafs and 1000 data points, and say 750 real data examples fall into leaf number one and remaining 250 examples fall into leaf number two. We would randomly choose leaf number two 25% of the time and leaf number one 75% of the time. Once we've chosen a leaf we begin generating data in the same naive way: We assume it is independently distributed in that leaf. Now

that the second batch of synthetic data is generated, still naive but slightly more refined, we create a new Random Forest Classifier whose task is again to discriminate between synthetic and real data. We interchangeably repeat the synthesis step and discrimination step each time using more refined data and a more refined classifier.

A simple trend can be observed that the classifier accuracy steadily falls after each iteration. The goal is to reach approximately 50% accuracy, which means that the classifier is no longer able to distinguish between real and synthetic data. This is not a guaranteed behavior, with improper treatment of the data this algorithm either fails to converge to the desired accuracy, or it achieves this accuracy but the results are not realistic.

2.1 Initial runs

Our initial runs with ARFs were naive, we performed next-to-none, data preparation and feature selection just to see how far we can stretch this algorithm. Suffice to say it did not go so well.

First, simply passing peptide data with no preprocessing severely impacts the models ability to converge; we suspect this is due to the fact that the peptide data stretches across several orders of magnitude. To remedy this issue, we attempted to scale the data using several scaling algorithms: Standard Scaling, Robust Scaling, and Quantile Transformation.

2.2 Scaling the data

Standard scaling simply scales the data to zero mean and unit variance, and is often used in classification problems. However, it is also a well-known fact that it is quite sensitive to outliers, which are often present in peptide features.

Robust scaling is aptly named due to its robustness to outliers, and it scales each example in the following way

$$\hat{x}_i = \frac{x_i - m}{IQR}$$

The symbol m represents the median of the data, and IQR stands for Inter Quartile Range. IQR is the span between the first and the third quartile, i.e. the difference between the example larger than 75% of the data and the example larger than 25% of the data.

Although this helped the model to converge, it caused some undesired effects, most notably the algorithm predicted negative abundances, which is not realistic for abundance measurements.

The preprocessing method that finally seemed to both give realistic abundances and resulted with the model converging was the Quantile Transformer. Instead of using the value of feature we're modeling, we use its quantile value. The quantile value is calculated in the following way: we order the examples from lowest to highest in value, and the quantile value of an example is the fraction of the data that is smaller in value than it. E.g., if an example is larger than 33% of the data its quantile value is 0.33. This ensures that each feature is uniformly distributed (this does not mean that the data as a whole is uniformly distributed).

2.3 Selecting the peptides

We are attempting to model data that has in total 21573 features and 5585 examples. While this is a fairly large number of examples for all sorts of learning algorithms, the

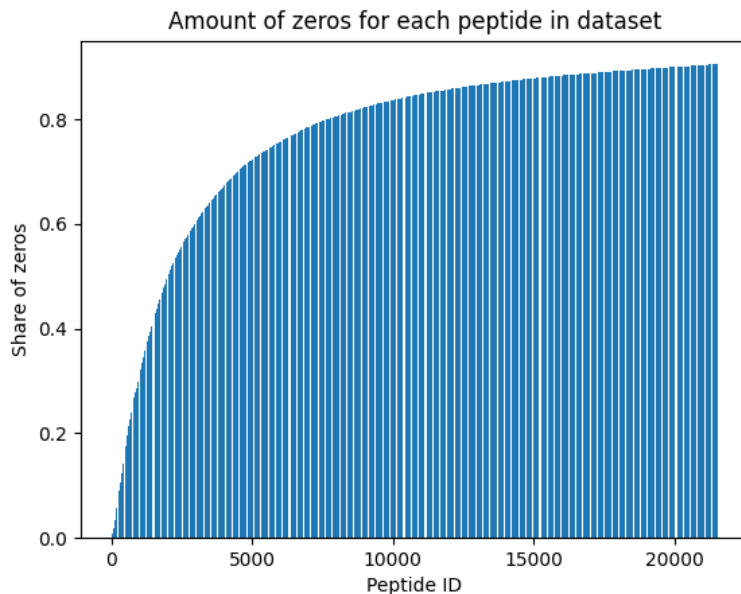


Figure 1: Peptide columns for real patients sorted in ascending order of zero shares. A zero share is calculated as the fraction of the full peptide data that is exactly zero.

number of presents its own set of problems. For ARFs this large number of features severely slowed down the training procedure, as well as data generation. More importantly the trained model failed to converge at desired accuracy levels.

An early observation we made was that plenty of peptides are often not present in patient data, i.e. their values are mostly clustered around zero. After some experimentation we’ve confirmed that the previously used 40% zero percentage threshold works best for ARFs as well.

With that in mind we calculated for each peptide the percentage of examples that are zero, if that number is less than the threshold we pass it to the ARF (or some other) model if not we perform a simpler data imputation method (more on that later).

Depending on whether this is one on single event data or on all data we get varying numbers for the number of peptides that get passed to the model, however, for the 40% zero percentage this number never exceeds more than 5% of all peptides, so the vast majority of peptides get imputed while only a minority gets modeled in some complex way.

Finally, this stabilized the ARF training procedure, and the model managed to converge. It’s worth noting that using both peptide selection and proper preprocessing (Quantile Transformer) was necessary for the model to work.

3 Histogram Imputation

There are several underlying assumptions justifying the splitting and imputation of peptide columns. First, only a smaller amount of peptides has sufficiently expressive data to extract their codependencies. The peptides that have expressive enough data are the ones below the 40% zero threshold. For the peptides above the threshold, this assumption predicates that either they hold no codependence with other peptides or the amount of data is not sufficiently large to extract them.

In the image 1 we can see the ratio of zeros rows to all rows for each peptide in a sorted

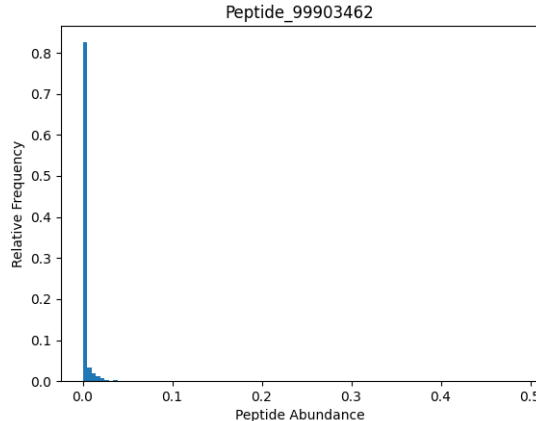


Figure 2: Histogram of abundance for Peptide 99903462.

fashion. It’s worth noting that the median of this distribution and center of mass is close to peptide ID 10000, which corresponds to about the 0.8 zero percentage threshold. While this may seem like there are plenty of peptides that are different than zero the reality is exactly the opposite. If we wish to work only with the peptides that have less than 50% zero values we would be working with only 10

In situations such as this, we are discouraged from using any sophisticated learning models, considering they will most likely over-fit the nonzero instances. To solve this problem we decided to generate these high-zero peptides independently from the rest of the data, without explicitly fitting any distribution to them. This leads us to the so-called Histogram Imputation. The procedure is a simple one, for each such peptide we group its values into bins (usually 100, but this number may vary). Each histogram will have a certain number of values in it and each histogram will cover some range of abundancies, that is, the bins will have a certain width. Once we’ve grouped abundance values in bins we take a look at each bin. For a certain bin we calculate the median of the values inside the bin (In some examples the bins can be very large, so a simple average value may be skewed towards larger values). Next we calculate the percentage of data that falls into the bin.

To generate the data we randomly choose a histogram bin, with the probability equal to the relative frequency of that bin. Once the bin has been chosen we input the median bin value as a synthetic data point. In the image 2 we can see a relative frequency histogram for a peptide with more than 80% zero counts. The first bin contains 80% of the data while the others each contain less than 5%. This means that the first bin (the one containing zeros) will be chosen 80% of the time, while others will be chosen according to their relative frequencies.

4 CTGAN report

In this section, we briefly discuss our approach based on CTGANs. GANs or Generative Adversarial Networks are a type of generative models that are based on two neural networks, one trained to generate synthetic data, and the other to discriminate between real and synthetic data. These two networks are trained in unison, one trying to *fool* the other. The training lasts until the networks achieve a so-called Nash equilibrium. While they are famous for generating high-fidelity images, GANs are notoriously difficult to train. The

entire process is often unstable, and the generating network can sometimes memorize exact instances of the training data. Furthermore, since one uses two loss functions instead of one, the value of these losses is no longer indicative of the network performance, and the above-mentioned Nash equilibrium can be obtained with neither network performing their task well.

Because of this, we were even more cautious with the increasing size of feature space, so making fine adjustments to the zero percentage threshold was necessary. Histogram imputation was again used for peptides that have a higher zero count than the desired threshold, while CTGANs were applied to the remaining peptides and clinical data (both numeric and categorical). Since Histogram Imputation was discussed in its own section, here we focus only on GANs and their fine-tuning. Again, numerical features were scaled using the Quantile Transformer.

4.1 Initial runs

Initially, we tested the CTGAN model with a very low zero threshold of 10%, i.e. only peptides with less than 10% of zero values were passed to the GAN model. This resulted in only 29 peptide columns generated with the GAN. Even in these initial runs, we’ve noticed that the model produces non-realistic results such as negative abundances. This is solved by enforcing minimum and maximum values of the data based on observations. While these restrictions are necessary for the model, they introduce a certain bias into the model.

4.2 Increasing the zero percentage threshold

Obviously, using such a zero percentage threshold as low as 10% will negatively impact the rest of the peptides, since they’re now being imputed independently via the Histogram Imputation. With this in mind, we attempted to increase the threshold to 30% which resulted in 594 peptides being passed to the GAN.

Several indicators that CTGANs were not ideal for this task arose. Most notably, the network treated sparse peptide abundance columns as discrete, i.e. categorical data, which they were not.

Secondly, to properly fit the data the model hidden layer size needed to be increased from 256 to 512 for both generator and discriminator networks.

4.3 Results

Even in the best of cases, after extensive hyperparameter tuning, CTGANs failed to produce high-fidelity synthetic data. However, some trends regarding marginal distributions were accurately captured by GANs. This can be seen in the two plots [3](#), [4](#) depicting real and synthetic data for peptides as well as tables [1](#), [2](#). An important thing to notice is that CTGAN seems to better capture the statistical properties of clinical data than peptide data. As well as the fact that some implicit real connections from the data were preserved, concretely systolic pressure was always larger than diastolic pressure.

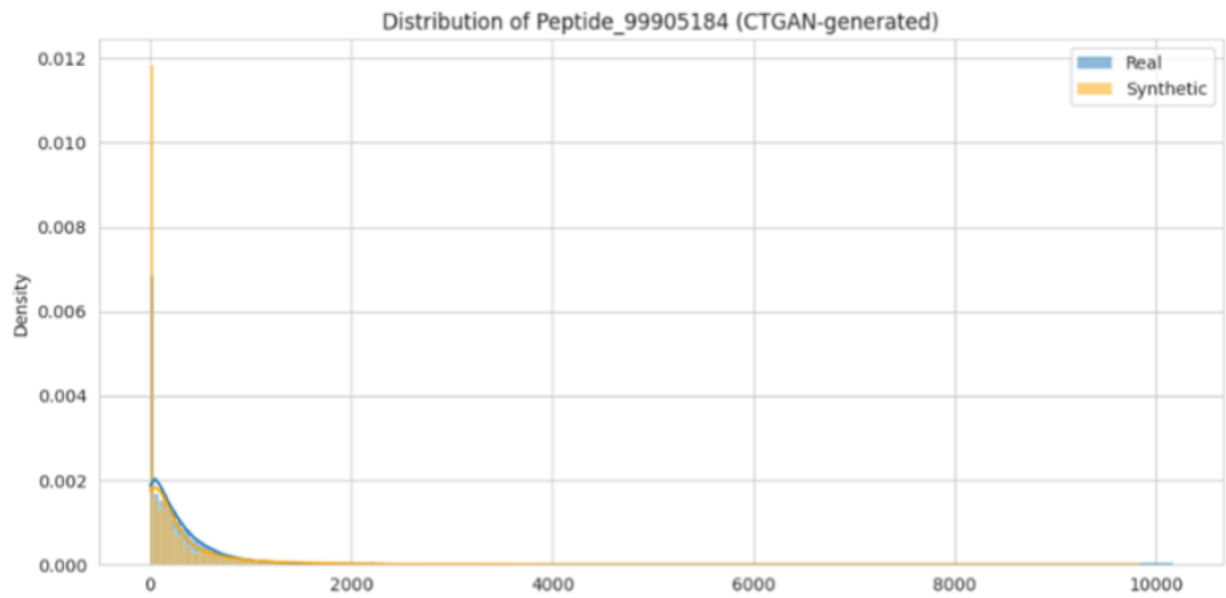


Figure 3: Comparison of marginal distributions of real and CTGAN generated data for Peptide 99905184.

Real Variable	Mean	Std. Dev.	Min.	Max.
GFR	76.81	25.90	5.64	155.50
Blood pressure Diastolic	78.23	9.95	37.00	130.00
Blood pressure Systolic	134.52	16.97	75.00	226.00
Peptide 99900054	61.94	78.85	0.00	762.60
Peptide 99900220	265.05	464.53	0.00	6876.54

Table 1: Real variable statistics.

Synthtetic Variable	Mean	Std. Dev.	Min.	Max.
Blood pressure Diastolic	78.80	12.25	37.16	129.59
Blood pressure Systolic	145.80	29.44	75.28	25.72
Peptide 99900054	67.00	109.48	0.00	751.47
Peptide 99900220	195.57	518.39	0.00	6818.90

Table 2: CTGAN generated variable statistics.

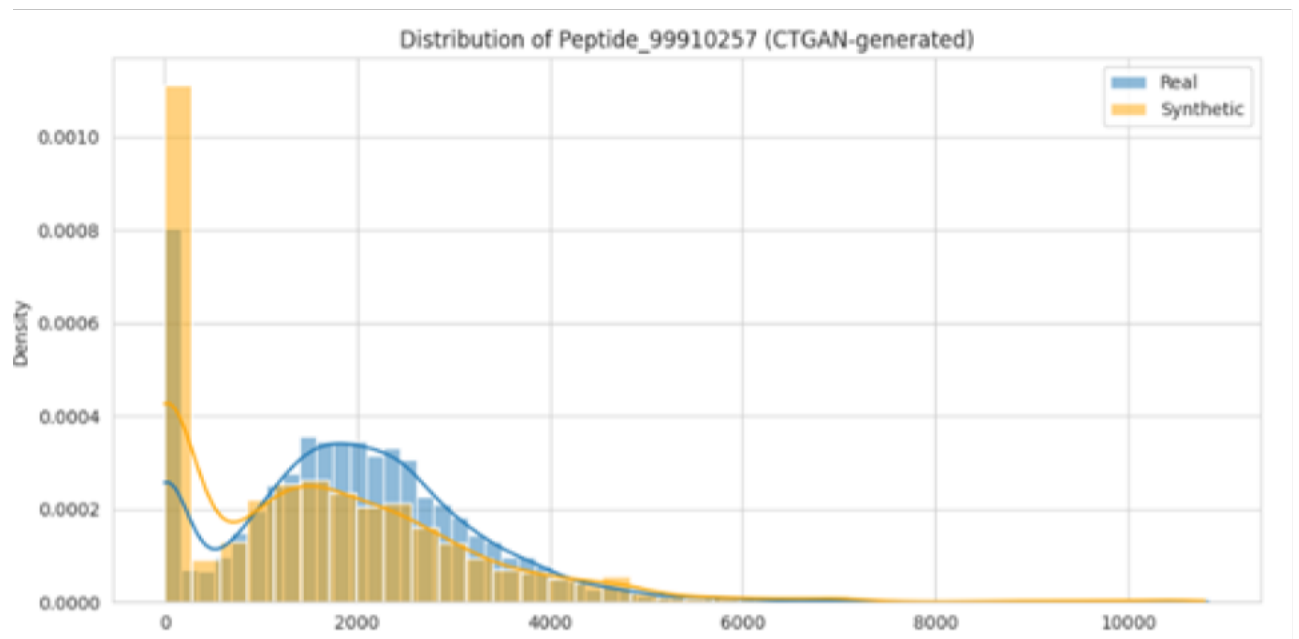


Figure 4: Comparison of marginal distributions of real and CTGAN generated data for Peptide 99910257.